Ensemble methods

Ensemble methods are machine learning techniques that **create multiple models (weak learners)** and then **combine their predictions** to achieve improved overall results.

- **Weak Learners:** Individual models used within an ensemble method. They are often simple and may perform only slightly better than a random guess on their own (e.g., individual classification and regression trees).
- **Goal:** To transform these weak learners into **strong learners** capable of making highly accurate predictions.

Key Characteristics:

- **Benefit:** Significantly **boosts prediction accuracy** (e.g., turning simple, less accurate trees into powerful predictors).
- **Drawbacks:**
  - **Increased Computation Time:** Training multiple models and combining them takes more processing power.
  - **Reduced Interpretability:** Combining many models makes it harder to understand how a single prediction was derived, compared to a single, simple model.

Examples:

- **Bagging:** An ensemble technique that trains multiple models (e.g., trees) independently on different subsets of the training data and then averages their predictions.
- **Random Forests:** An extension of bagging that builds multiple decision trees by introducing additional randomness (e.g., random selection of features for splitting), and then aggregates their results. These methods combine numerous trees for dramatic accuracy improvements.

Instability of Trees (High Variance)

The main drawback of individual decision trees is their **instability**, meaning they have **high variance**. One major reason for this instability is that if a split changes, all the splits under it change as well, thereby propagating the variability. By using ensemble methods, we can greatly improve prediction accuracy, but we suffer in terms of interpretability.

The Bootstrap

The bootstrap is a powerful statistical technique that involves random sampling with replacement from the original dataset to create multiple new training datasets.

Bootstrap is a resampling method that:

- Takes the original dataset of size n.
- Creates new datasets (called **bootstrap samples**) of the **same size** n.
- Each bootstrap sample is formed by **randomly selecting data points with replacement**, so the same data point can appear multiple times in one sample.
- **Bootstrap Sample:** A sample of size n drawn **uniformly at random with replacement** from the original training data $(x_1, y_1), \ldots, (x_n, y_n)$.
    - This means some original data points may appear multiple times in a bootstrap sample, while others may not be included at all.

Bootstrap Error Calculation ($Err^{\hat{Boot}}$):

A common way to estimate prediction error using bootstrap samples is:
$Err^{\hat{Boot}} = \frac{1}{B}\frac{1}{n}\sum_{b=1}^{B}\sum_{i=1}^{n}(y_i - \hat{f}^b(x_i))^2$ Where:

- B is the total number of bootstrap samples.
- $\hat{f}^b(x_i)$ is the prediction of observation x_i made by the model trained on the b-th bootstrap dataset.

Out-of-Bag (OOB) Observations:

For a given bootstrap sample, the observations from the original training data that were **not included** in that specific sample are called OOB observations.

- For large n, approximately $1/3$ **(or** $e^{-1} \approx 0.368$**)** of the original observations are expected to be OOB in any given bootstrap sample.

Problems and Improvements:

- **Problem:** $Err^{\hat{Boot}}$ is often **too optimistic** because there's a large overlap (approximately 63.2%) between the training set used for a model and the data points used for its error calculation within this formula.
- **Possible Improvements:**
    1. **Using only OOB Observations:** Calculate the error only for those observations that were **not included** in the specific bootstrap dataset used to train the model. This is more reliable as these OOB points act as a "test set" for that particular bootstrap model.

Bagging (Bootstrap AGGregation)

Bagging is an ensemble learning technique that creates multiple models trained on different bootstrap samples (random samples with replacement) of the original dataset and gives a final prediction by combining the predictions of all the models. It primarily reduces variance (improves stability) and helps to prevent overfitting, especially for high-variance models like decision trees.

How Bagging Trees Works:

The procedure for bagging applied to decision trees (Bagging Trees) involves three main steps:

1. **Generate Bootstrapped Training Sets:** Create B different bootstrap samples from the original training data. Each bootstrap sample is of the same size n as the original data and is drawn with replacement.
   - Example: $(x_1^b, y_1^b), (x_2^b, y_2^b), \ldots, (x_n^b, y_n^b)$ for $b = 1, \ldots, B$.
2. **Fit Individual Trees:** For each of the B bootstrapped training sets, fit an independent regression tree ($\hat{f}^b$) or a classification tree ($\hat{c}^b$). Each tree is typically grown to maximum depth without pruning (to ensure high variance, which bagging aims to reduce).
3. **Combine Predictions:**
   - **For Regression Trees:** The final prediction $\bar{f}(x)$ for a new observation x is the **average** of the predictions from all individual trees: $\bar{f}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x)$
   - **For Classification Trees:** The final prediction $\bar{c}(x)$ for a new observation x is determined by **majority vote (the Mode)** among the predictions of all individual trees: $\bar{c}(x) = \text{Mode}\{\hat{c}^b(x), b = 1, \ldots, B\}$ (this is also called consensus).

Benefit: By averaging or majority voting predictions from many different trees (each trained on a slightly different version of the data), Bagging significantly **reduces the overall variance** of the ensemble prediction, leading to improved accuracy.

Bagging estimates of class probabilities

🎯 Goal:

In a classification problem, instead of just predicting **which class** an input x belongs to, we sometimes want to know **how likely** it is to belong to each class — these are called **class probabilities**.

🧠 Standard Bagging for Classification:

- You build **many trees**, each on a different **bootstrap sample**.
- For a new input x, each tree gives a **class label** (e.g., "cat", "dog", etc.).
- The final prediction is the **majority vote** — the class that got the most votes.

✅ This is **simple voting**.

📊 Estimating Class Probabilities (Improved Bagging):

Instead of just counting which class wins, we go one step further:

1. **Each tree gives not just a class**, but also the **probabilities** of the classes.

For example, one tree might say:

Class	Probability
Dog	0.7
Cat	0.2
Rabbit	0.1

These probabilities are based on the **proportion of training samples** in the same region of the tree.

2. We do this for **each tree.**

3. Then we **average** the probabilities from all trees:

$$\bar{p}_k(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_b^k(x)$$

- $\hat{p}_b^k(x)$: the probability of class k from tree b.
- $\bar{p}_k(x)$: the **average predicted probability** for class k over all B trees.

4. The final predicted class is the one with the **highest average probability**:

$$\hat{f}\,(x) = \arg\max_k \bar{p}_k(x)$$

🔍 Why is this better?

- It gives you **probability estimates**, not just hard class labels.
- These probabilities can be useful if:
  - You want to measure **uncertainty**.
  - You care about **confidence** in predictions.

    - You want to adjust thresholds (for example in medical diagnosis).
- According to the **ESL book**, this **improved bagging method** sometimes even results in **better classification accuracy**.

✅ Summary (in Plain English):

Method	What it does	Final Prediction
Standard Bagging	Each tree votes for a class.	Class with most votes.
Improved Bagging	Each tree gives class probabilities. We average them.	Class with highest average probability.

Why Does Bagging Work?

Bagging works by **reducing the variance** of high-variance models like decision trees. It does this by training multiple models on different **bootstrap samples** (samples with replacement) and combining their predictions through **averaging (regression)** or **majority voting (classification)**.

If each individual model has an error rate less than 50% and their errors are **uncorrelated**, the majority vote becomes increasingly accurate as the number of models increases. This is because random errors tend to cancel out when averaged.

In practice, models are not fully independent, so the error does not go to zero, but bagging still improves **stability** and **generalization** by reducing model variance.

🧠 The Setup (Recap):

We have:

- **K = 2** classes: class **1** (true class) and class **2** (wrong).
- **B classifiers**: each is trained on a **different bootstrap sample**.
- Each classifier $\hat{c}_b(x)$ makes a **wrong prediction** (class 2) with probability 0.4 — that means it's a **weak but better-than-random** classifier.

We define:

$$Z = \sum_{b=1}^{B} I\{\hat{c}_b(x) = 2\}$$

That is: the **number of classifiers** that vote for the **wrong class**.

Since each makes a mistake with probability 0.4, and they're assumed **independent**, $Z \sim \text{Binomial}(B, 0.4)$.

Then, bagging takes the **majority vote**. So the ensemble makes a mistake **only if more than half of the classifiers vote wrong**:

$$\text{Bagging error} = P(Z \geq (B+1)/2)$$

Now here's the **key observation**:

- As $B \to \infty$, this probability → 0.
- So, in theory, bagging gives **perfect accuracy** if:
  - The individual classifiers are **better than random** (error < 50%),
  - And they are **independent**.

🔍 Why Does Bagging Work?

Because of the **law of large numbers** + **voting**:

> Even if individual classifiers are weak (like 60% accurate), the **majority vote becomes stronger and stronger** when you average many of them — **as long as their mistakes are not too correlated**.

That's the beauty of ensemble learning: **weak learners + diversity = strong learner**.

⚠️ So Why Doesn't It Become Perfect in Practice?

❌ Independence is an unrealistic assumption.

In practice:

- The classifiers are **not truly independent**.
- They are trained on bootstrap samples from the **same dataset**.
- So their predictions are **correlated** — they often make mistakes on the **same examples**.

This correlation **limits the gain** from bagging.

📉 What Happens in Real Life:

- Initially, as we increase B, the error rate **drops**.
- But because the classifiers are **similar**, after a point, adding more trees **doesn't help much**.
- The test error **levels off** — we reach a limit set by the **bias + correlation** in the models.

This is exactly what the **orange vs. green curves** in ESL are showing:

- **Orange**: regular bagging (voting).
- **Green**: probability averaging (improved bagging).
- Both improve with more trees, but **don't go to 0**, because of correlation.

✅ Final Summary:

Concept	Meaning
Why bagging works	Majority voting averages out individual model errors — especially when classifiers are independent.
What's required	Individual models must be better than random, and ideally **uncorrelated**.
Real-world limitation	Models trained on bootstrap samples from the same data are **not independent**, so improvement **slows down**.
Bottom line	Bagging reduces variance and helps generalization — but can't eliminate error entirely due to model correlation.

When Does Bagging Fail?

Bagging fails when the **base classifiers are consistently poor**, i.e., each has a **misclassification rate greater than 50%**. In this case, even with independent models, majority voting will amplify the wrong decisions. As the number of classifiers $B \to \infty$, the bagged classifier will almost surely predict the **wrong class**, leading to **perfect inaccuracy**.

This happens because:

$$\Pr(\hat{c}_b(x) = \text{wrong class}) = 0.6 > 0.5 \Rightarrow \Pr(\text{majority vote wrong}) \to 1 \text{ as } B \to \propto$$

Even though in practice classifiers are not independent, the key idea is:

> **Bagging improves good (better-than-random) classifiers, but worsens bad (worse-than-random) classifiers.**

Disadvantages of Bagging

1. **Loss of Interpretability** Bagging aggregates predictions from many models (e.g., trees), so the final classifier is no longer a single decision tree. This results in a **loss of interpretability**, which is a key advantage of individual trees.
2. **Increased Computational Cost** Bagging requires training B models (often decision trees), which is **computationally expensive**, especially if each tree is pruned or validated. The training time scales linearly with B.

3. **Limited Model Space** While bagging expands the model space from a single model to an ensemble, it still has **limitations**. For example:

   - Bagging decision trees may struggle to approximate **linear or diagonal decision boundaries**.
   - If the base learners (like axis-aligned trees) are poorly suited to the data geometry, bagging won't fix the underlying **bias**.

   Example (ESL, p. 288): For data where the true decision boundary is linear, such as $x_1 + x_2 = 1$, bagged trees (even with 50 trees) fail to accurately model the boundary. The ensemble's decision surface is curved and misaligned, and test error remains relatively high (e.g., 0.166).

✅ Summary (for bullet points)

- ❌ Loss of interpretability (no longer a single tree)
- ⏳ High computational cost (training many models)
- 🧱 Model space still limited (can't learn linear/diagonal boundaries well with axis-aligned trees)
- 📉 Bagging cannot fix poor bias in weak learners — it mainly reduces **variance**, not **bias**

Out-of-Bag (OOB) Error

Out-of-bag (OOB) error is a way to estimate the **test error** of a **bagging** model **without needing a separate validation set or cross-validation**.

🧠 How It Works:

In **bagging**, each base model is trained on a **bootstrap sample** of the training data:

- A bootstrap sample is created by sampling **with replacement** from the original dataset.
- On average, about **63%** of the data points appear in any given bootstrap sample.
- The remaining **~37%** of the data (not included in that sample) are called the **out-of-bag (OOB) samples**.

🔁 Using OOB for Error Estimation:

The OOB error is calculated for each original observation x_i as follows:

1. For each original observation x_i, identify **all** the bootstrap samples b for which x_i was **out-of-bag** (i.e., not used to train the model $\hat{f}^b$).
2. Use these specific $\hat{f}^b$ models to make a prediction for x_i.
3. **Combine these OOB predictions for x_i:**

- For **regression**, average the predictions $\hat{f}^b(x_i)$ from all models where x_i was OOB.
- For **classification**, take a majority vote of the class predictions $\hat{c}^b(x_i)$ from all models where x_i was OOB.

4. Calculate the error (e.g., squared error for regression, misclassification for classification) for x_i based on this combined OOB prediction and its true label y_i.
5. Finally, the overall OOB error is the **average of these individual errors** across all n original observations.

✅ Advantages of OOB Error:

- Acts like **built-in cross-validation** — no need to hold out a validation set.
- Provides a **reliable estimate** of test error, especially for bagging-based models like **random forests**.
- Efficient — it reuses the training process, so no extra computation is needed.

📝 Summary (Exam-Ready)

> **Out-of-bag (OOB) error** is the prediction error estimated using only those models that did **not** include a given training point in their bootstrap sample. It provides an unbiased estimate of the model's test error and avoids the need for separate validation or cross-validation.

Random forests

Random Forests is an extension of Bagging that combines multiple decision trees to improve prediction accuracy and control overfitting.

The key distinction from standard Bagging lies in how they vary the training data:

- **Bagging:** Varies the **rows** of the training set by randomly drawing observations *with replacement* (bootstrap samples).
- **Random Forests:** Varies **both the rows AND the columns (predictors)** of the training set.

Key Mechanism: Random Subset of Predictors

The crucial element that creates this additional variation is a **tuning parameter** called m:

- **Tuning Parameter (m):** Before *each* split in the process of growing an individual tree, the algorithm randomly selects a subset of m predictors (features) from the total p available predictors as candidates for that specific split. The best split is then chosen only from these m selected predictors.

Typical Values for m:

The choice of m significantly impacts the correlation between the trees and thus the overall performance. Common rules of thumb are:

- **For Classification:** $m = \sqrt{p}$ (square root of the total number of predictors).
- **For Regression:** $m = p/3$ (one-third of the total number of predictors).

Relationship to Bagging:

- If you set $m = p$ (meaning all predictors are considered at each split), Random Forests effectively become equivalent to **Bagging** as a special case. This highlights that Random Forests are a generalization of Bagging, adding an extra layer of randomness.

Why Do Random Forests Work?

Random Forests improve upon bagging by introducing **randomness in feature selection**, which helps **decorrelate** the trees in the ensemble.

🔍 Key Idea:

In bagging, even though each tree is trained on a different bootstrap sample, they often use the **same strong predictors** to make splits. This leads to **correlated trees**, which **limits the variance reduction** from averaging.

Random Forests fix this by:

- **Randomly selecting a subset of features** at each split (instead of using all features).
- This **reduces the correlation** ρ between trees, making their errors more independent.

🧠 Mathematical Insight:

If we have B identically distributed trees $Z_1, Z_2, \ldots, Z_B$, each with:

- Mean μ
- Variance σ^2
- Pairwise correlation ρ

Then the **variance of the average prediction** from the ensemble is:

$$\mathrm{Var}(\bar{Z}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

✅ This shows:

- If ρ is small, **variance drops more** as we increase B.
- Random Forests aim to **reduce** ρ without increasing σ^2 too much.

📌 Conclusion:

> **Random Forests work because they reduce the correlation between trees**, leading to **better variance reduction** when their predictions are averaged. This improves accuracy and stability over standard bagging.

randomForest() algorithm

The **Random Forest** algorithm builds an ensemble of decision trees using bootstrapping and random feature selection to improve prediction accuracy and reduce overfitting.

🔁 Step-by-Step Algorithm:

1. **Input**:
   - Training dataset with n observations and p predictors.
   - Number of trees to grow: `ntrees`.
   - Number of variables to consider at each split: `m` (usually $m < p$).
2. **For** $i = 1$ **to** `ntrees`:
   A. **Draw a bootstrap sample** (sample with replacement) from the training data.
   B. **Grow a regression tree** to this sample without pruning.
   C. At each node of the tree:
      - **Randomly select** m **variables** from the full set of p predictors.
      - **Find the best split** among the m variables (based on criteria like minimizing mean squared error).
      - **Split the node** into two child nodes accordingly.
   D. Continue splitting until a stopping criterion is met (e.g., minimum number of observations in a node).
3. **Repeat** until all `ntrees` trees are built.
4. **Prediction** for a new input x:
   - Pass x through all trees.
   - Take the **average of the individual tree predictions**.

📌 Important Notes:

- Trees are **not pruned**.
- Each tree is **different** due to:
  - Different bootstrap samples.
  - Different feature subsets at each split.
- Averaging over many uncorrelated trees **reduces variance**.

In [ ]: